🔐 qec_martin — Quantum Error Correction & Elliptic Curve Cryptography

> A high-performance Rust toolkit for **Quantum Error-Correcting Codes (QEC)** and **Elliptic Curve Cryptography (ECC)**, featuring lattice-based encryption, quantum circuit simulation, and benchmarking infrastructure.

✨ Features

Module	Description
Quantum ECC	Error-correcting code construction and evaluation for quantum computing
LWE Encryption	Lattice-based Learning With Errors (LWE) post-quantum encryption
Elliptic Curves	Weierstrass curve point addition and scalar multiplication
Circuit Builder	Binary targets for building and evaluating quantum circuits
Superfast Encryption	High-throughput encryption primitives with AES-GCM & SHA-3
Benchmark Suite	Automated performance benchmarking with TSV/JSON output

---

Tech Stack

- **Language:** Rust 2021 Edition
- **Crypto:** `k256` (ECDSA), `aes-gcm`, `sha3`, `alloy-primitives`
- **Math:** `ruint`, `rand`, `rand_core`
- **Compression:** `zstd`
- **CI/CD:** GitHub Actions with auto-fix (`cargo fmt` + `clippy`)
- **Pre-commit:** Hooks for local linting enforcement

🚀 Quick Start

Prerequisites

```
# Install Rust (if not already installed)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Clone & Build

```
git clone https://github.com/elon00/qec_martin.git
cd qec_martin/quantum_ecc_add

# Build release binary
```

Generated by Kimi.ai

```
cargo build --release

# Run tests
cargo test --all-features
```

Available Binaries

```
# Build a quantum circuit
cargo run --bin build_circuit

# Evaluate a quantum circuit
cargo run --bin eval_circuit

# Run superfast encryption demo
cargo run --bin superfast_encryption
```

📁 Project Structure

```
qec_martin/
├── .github/
│   └── workflows/
│       ├── rust-auto-fix.yml   # Auto-format & clippy fix on every push
│       ├── rust-ci.yml         # Build, test, lint checks
│       └── stale.yml           # Stale issue management
├── quantum_ecc_add/
│   ├── src/
│   │   ├── lib.rs              # Library entry point
│   │   ├── main.rs             # CLI entry point
│   │   ├── circuit.rs          # Quantum circuit logic
│   │   ├── sim.rs              # Quantum simulation engine
│   │   ├── lwe_encryption.rs   # Lattice-based encryption
│   │   ├── weierstrass_elliptic_curve.rs  # ECC operations
│   │   ├── bin/
│   │   │   ├── build_circuit.rs
│   │   │   └── eval_circuit.rs
│   │   └── point_add/          # Point addition utilities
│   ├── Superfast_Encryption.rs # Standalone encryption binary
│   ├── Cargo.toml              # Crate manifest
│   ├── rustfmt.toml            # Formatting rules
│   ├── benchmark.sh            # Benchmark automation script
│   ├── benchmark.json          # Benchmark configuration
│   └── results.tsv             # Benchmark results
├── .pre-commit-config.yaml     # Local pre-commit hooks
└── README.md                   # This file
```

🔧 Development Workflow

Auto-Fix (GitHub Actions)

Every push automatically triggers:

1. `cargo fmt --all` — Formats all Rust code
2. `cargo clippy --fix` — Applies safe lint fixes
3. Auto-commits fixes back to the branch

Pre-commit Hooks (Local)

Before any local commit, hooks verify:

```
# Install hooks once
pre-commit install

# Run manually
cd quantum_ecc_add
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
```

Formatting Rules

See `quantum_ecc_add/rustfmt.toml`:

- Max width: 100 characters
- 4-space indentation
- Unix newlines
- Auto-reorder imports & modules

📊 Benchmarking

```
cd quantum_ecc_add
chmod +x benchmark.sh
./benchmark.sh
```

Results are written to `results.tsv` and `benchmark.json`.

🤝 Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes — pre-commit hooks will auto-check formatting
4. Push to the branch — GitHub Actions will auto-fix any remaining issues
5. Open a Pull Request

Generated by Kimi.ai

> **Note:** The CI pipeline enforces `cargo fmt` and `cargo clippy`. Any formatting issues are automatically resolved by the `rust-auto-fix` workflow.

📄 License

This project is licensed under the MIT License. See `NOTICE` for attribution details.

🔗 Related

- Rust Book
- Cargo Clippy
- Quantum Error Correction
- Learning With Errors

Generated by Kimi.ai